Noise-Robust Speech Recognition with Word-Level Error Flagging: A Study Using Noise-Augmented Training



GROUP 9

Milestone 3: Architecture

Course: DS-AI Lab

Term: October 2026

1. Introduction

This report documents the dataset organisation, preprocessing, model architecture, and end-to-end pipeline for our noise-robust speech recognition system. The system takes a 16 kHz mono waveform and outputs a character-level transcript using a pretrained Wav2Vec 2.0 encoder (`facebook/wav2vec2-base-960h`) with a CTC head. Training data is a LibriSpeech subset that we corrupted with synthetic white, pink and brown noise at four signal-to-noise ratios (SNRs).

The pipeline runs end to end: dataset creation, leakage-checked splits, batching, training, test-set prediction, per-condition evaluation and single-file inference all execute without errors on the full 13,000-sample dataset. Key results from the executed notebooks:

Item	Value
Original recordings	1,000 (LibriSpeech train-clean-100, seed 42)
Generated samples	13,000 (1 clean + 12 noisy variants per recording)
Split (train / validation / test)	9,750 / 650 / 2,600 samples (750 / 50 / 200 recordings)
Model	Wav2Vec2ForCTC, 94,396,320 parameters, vocabulary of 32
Test WER / CER	25.12% / 15.89%
WER on clean audio	2.43%
WER on white noise at 0 dB	94.16%

2. Dataset Organisation and Preprocessing

2.1 Source datasets

The current dataset uses LibriSpeech speech plus synthetically generated coloured noise. MUSAN and ESC-50 were profiled and are reserved for the next stage, where realistic and unseen noise types will be introduced.

2.2 How the dataset is organised

1. 1,000 utterances were randomly selected (seed 42) from the 28,539 usable train-clean-100 utterances.
2. The recordings were split at the recording level (75% / 5% / 20%) into 750 train, 50 validation and 200 test recordings. Splitting happens before augmentation, so no recording and none of its noisy variants can appear in two splits.
3. Every recording produces 13 samples: 1 clean, plus 3 noise types (white, pink, brown) at 4 SNRs (20, 10, 5, 0 dB).

4. Each sample is stored as a PyTorch tensor (.pt, shape [1, N], float32) with a .txt transcript, and one row is appended to metadata.csv.

Split	Recordings	Samples	Per noise type (white / pink / brown)	Clean
Train	750	9,750	3,000 each	750
Validation	50	650	200 each	50
Test	200	2,600	800 each	200
Total	1,000	13,000	4,000 each	1,000

Each SNR level (20, 10, 5, 0 dB) has 3,000 samples overall; clean samples carry SNR = inf. Each test condition (noise type x SNR) contains 200 samples, one per test recording. The full metadata of the dataset is present in metadata.csv file. The data is arranged as follows :

metadata.csv columns: audio_id, audio_path, text, duration, original_sample_rate, noise_type, snr_db, split.

Audio IDs follow {speaker}_{chapter}_{utterance}_{noise}_snr{level}, for example 7190_90542_7190-90542-0043_white_snr20 (15.285 s, original rate 16,000 Hz).

2.3 Directory structure

Raw data (standard LibriSpeech layout, one .trans.txt per chapter):

```
LibriSpeech/train-clean-100/
├── <speaker_id>/
│   └── <chapter_id>/
│       ├── <speaker>-<chapter>-<utt>.flac
│       └── <speaker>-<chapter>.trans.txt
musan/{music, speech, noise}/       # noise source for later stages
ESC-50-master/audio/*.wav           # held-out noise for later stages
```

Processed data, with training, validation and test splits:

```
noisy_librispeech/
├── metadata.csv                    # 13,000 rows
├── augmentation/
│   └── noise_params.json           # SNR levels, noise types, sample rate
├── train/                          # 9,750 .pt + 9,750 .txt
│   ├── 7190_90542_7190-90542-0043_clean.pt / .txt
│   ├── 7190_90542_7190-90542-0043_white_snr20.pt / .txt
│   └── ... (pink / brown x snr 20, 10, 5, 0)
├── valid/                          # 650 samples (called "validation" in training code)
└── test/                           # 2,600 samples
```

Training outputs:

```
wav2vec2_librispeech/
├── checkpoint-*/                # last 2 checkpoints (save_total_limit=2)
├── final_model/                 # model weights + processor
├── test_predictions.csv
├── test_metrics.json
├── wer_by_noise_condition.csv
└── experiment_summary.json
```

2.4 Dataset Preprocessing steps

Stage A: dataset creation (offline, run once)

Step	Operation	Detail
1	Load audio	`soundfile` reads the FLAC file; transcripts come from the chapter `.trans.txt` files
2	Resample	`torchaudio` Resample to 16 kHz if the source rate differs
3	Mono conversion	Multi-channel audio averaged to a single channel
4	Length cap	Truncated to 30 s
5	Transcript normalisation	Lowercase, remove everything except `a-z`, `0-9` and spaces, collapse whitespace; empty transcripts are skipped
6	Noise synthesis	White: Gaussian. Pink: PSD proportional to 1/f. Brown: PSD proportional to 1/f^2. Generated via FFT shaping, DC removed, scaled to unit RMS
7	SNR mixing	Noise power scaled to P_signal / 10^(SNR/10) and added to the speech; no clipping so the SNR stays exact
8	Reproducibility	Per-sample seed derived from the global seed, audio ID, noise type and SNR
9	Storage	Waveform saved as `[1, N]` float32 `.pt`, transcript as `.txt`, one row appended to `metadata.csv`

Stage B: training-time preprocessing (per batch)

Step	Operation	Detail
1	Metadata cleaning	Resolve each `audio_path` to an existing file; map `valid` to `validation`; drop rows with missing audio, empty text or unknown split (0 of 13,000 dropped)
2	Leakage check	The `_clean` / `_{noise}_snr{X}` suffix is stripped to recover the source recording; the code asserts that no source appears in

Step	Operation	Detail
		more than one split. Passed, with 1,000 unique recordings
3	Waveform loading	`torch.load`, cast to float32, squeeze to 1-D mono, replace NaN values
4	Feature extraction	`Wav2Vec2Processor` (16 kHz) applies zero-mean, unit-variance normalisation to the raw waveform. No spectrogram is computed; the model consumes raw samples
5	Tokenisation	Character-level CTC tokenizer. Text is upper-cased and mapped to IDs; the space becomes the word delimiter token `|`
6	Batch collation	`DataCollatorCTCWithPadding` pads waveforms to the longest in the batch and pads labels with `-100`, so padded positions are ignored by the loss

Two details worth knowing. First, `processor.decode` collapses repeated characters by default (the CTC rule), so a label such as PROFESSOR displays as PROFESOR; the stored label IDs are intact, and evaluation decodes references with `group_tokens=False`. Second, the dataset stage removes apostrophes (`a-z0-9` only), so words like "don't" become "dont"; the evaluation-side normaliser would keep apostrophes but has nothing to keep. This affects reference text only slightly.

3. Model Architecture

3.1 Overview

The model is `Wav2Vec2ForCTC` initialised from `facebook/wav2vec2-base-960h`, a Wav2Vec 2.0 BASE encoder pretrained with self-supervised learning and fine-tuned for character-level CTC on 960 hours of LibriSpeech. It has 94,396,320 parameters, a vocabulary of 32 tokens and an expected sampling rate of 16 kHz.

Component	Configuration (BASE)	Role
Feature encoder	7 temporal conv layers, 512 channels, kernels (10,3,3,3,3,2,2), strides (5,2,2,2,2,2,2), GELU, group norm in layer 1	Turns the raw waveform into latent frames; total stride 320 samples (20 ms), receptive field 400 samples (25 ms)
Feature projection	LayerNorm, Linear 512 to 768, dropout	Maps latent frames to the transformer width
Positional encoding	Grouped convolutional embedding (kernel 128, 16	Injects relative position information

Component	Configuration (BASE)	Role
	groups)	
Transformer encoder	12 layers, hidden size 768, 12 attention heads, feed-forward size 3072	Builds contextualised frame representations; self-attention lets each frame use context from the whole utterance
Masked-spec embedding	Learned 768-d vector	Replaces masked frames in SpecAugment-style time masking during training. It was missing from the checkpoint and newly initialised
CTC head (`lm_head`)	Dropout, Linear 768 to 32	Per-frame distribution over characters
Decoding	Greedy argmax, collapse repeats, remove blank	Produces the transcript; `|` becomes a space

The vocabulary has 32 symbols: `<pad>` (which doubles as the CTC blank), `<s>`, `</s>`, `<unk>`, the word delimiter `\|`, the letters A to Z, and the apostrophe.

3.2 Data flow

The diagram in Section 3.4 shows the full path from a raw FLAC file to the final transcript.

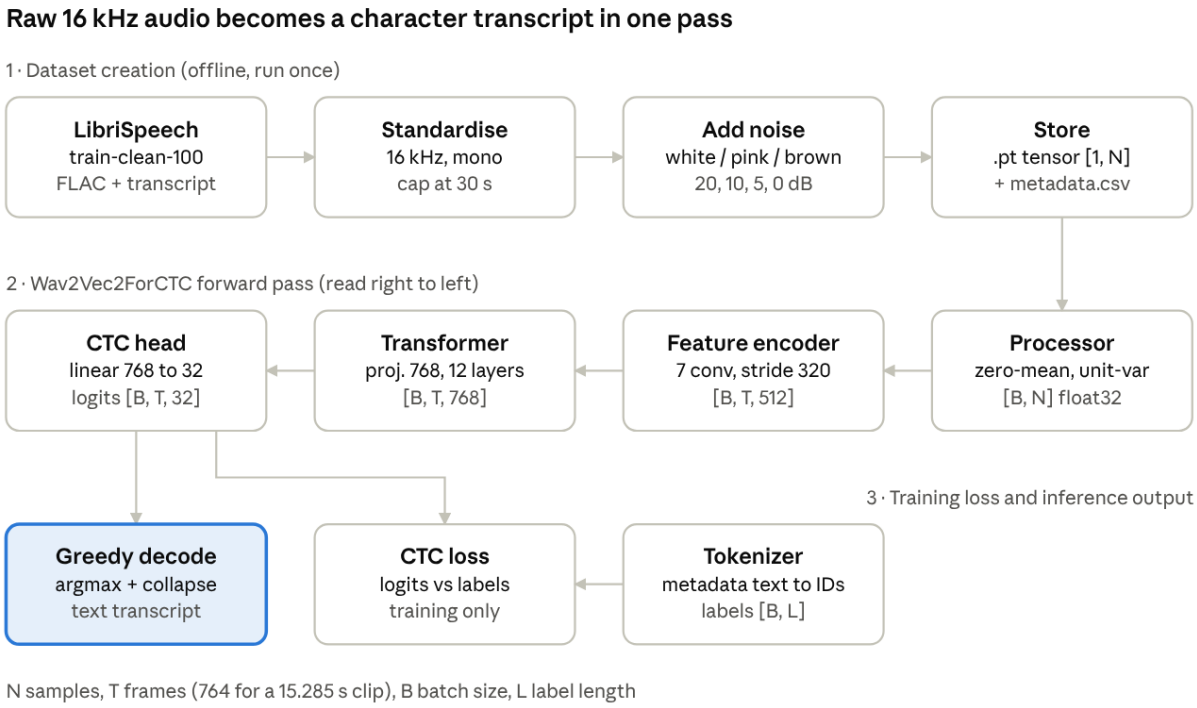
3.3 How the processed data matches the model input

Tensor	Shape and dtype	Notes
Stored waveform (`.pt`)	`[1, N]`, float32	N = duration x 16,000
`input_values` (one sample)	`[N]`, float32	Normalised; observed `[244560]` for a 15.285 s clip
`input_values` (batch)	`[B, N_max]`, float32	Zero-padded to the longest clip; observed `[2, 244560]`
`labels` (batch)	`[B, L_max]`, int64	Character IDs, padded with `-100`; observed `[2, 258]`
Encoder frames	`[B, T, 768]`	T = 764 for 244,560 samples; 49 frames per second of audio
`logits`	`[B, T, 32]`	One distribution per 20 ms frame

Frame counts follow from the convolution stack: 244,560 samples give 764 frames, and the 30 s maximum (480,000 samples) gives 1,499 frames. CTC needs at least as many frames as output characters; the example clip has 764 frames for 258 characters, and LibriSpeech speech (about 15 to 17 characters per second against 49 frames per second) leaves ample headroom.

Training configuration: per-device batch size 2 with 4 gradient-accumulation steps (effective batch 8), AdamW, learning rate 3e-4 with 500 warm-up steps and linear decay, weight decay 0.005, 2,000 steps, fp16, gradient checkpointing, evaluation and checkpointing every 250 steps, best checkpoint selected by lowest validation WER. Hardware was a Tesla T4 (PyTorch 2.11.0+cu130); the run took 6,192 s (about 1 h 43 min).

3.4 Architecture diagram



The top row prepares the data offline. The model turns each waveform into one score vector per 20 ms frame, which feeds the CTC loss during training and greedy decoding at inference.

4. Architecture Justification

4.1 Why Wav2Vec 2.0 with CTC fits this problem

- **Operates on raw audio.** The convolutional front end learns its own features, so the model can adapt to noise rather than depend on hand-designed log-mel features that were tuned for clean speech.

- **Self-supervised pretraining.** Representations learned from large amounts of unlabelled speech transfer well to small labelled sets, which matches our setting of about 1,000 recordings.
- **Frame-level outputs.** CTC gives a posterior over characters for every 20 ms frame. This supports the project's second goal, word-level error detection, because per-frame confidence and character timings can be aggregated into word-level scores without a separate aligner.
- **Non-autoregressive and efficient.** A single forward pass yields the whole transcript; there is no token-by-token decoding loop, so inference is fast and predictable.
- **Easy to adapt.** The Hugging Face implementation provides the processor, tokenizer, CTC loss and Trainer integration, which kept the notebook short and reproducible.

4.2 Strengths and limitations

Strengths	Limitations
Strong accuracy on clean read English (2.43% WER on our clean subset)	Degrades sharply in broadband noise: 94.16% WER for white noise at 0 dB
Raw-waveform input and context from self-attention	Conditional independence of CTC outputs; no built-in language model, so spelling errors such as IMBRYO or COMPARITO appear in noise
Compact (94 M parameters), trainable on a single T4	Attention cost grows quadratically with length; inputs are capped at 30 s
Per-frame posteriors support word-level confidence	BASE checkpoint has no attention mask, so padded batches use zero padding without masking
Fixed 16 kHz input and character vocabulary make the pipeline simple	English-only vocabulary; no digits or punctuation

5. End-to-End Pipeline

5.1 Pipeline stages and verification

The notebooks execute every stage; the table lists what was checked and the observed output.

Stage	Check	Observed result
1. Dataset generation	1,000 recordings processed, 0 skipped	13,000 samples written (9,750 / 650 / 2,600)
2. Metadata	All rows load and resolve to existing files	13,000 of 13,000 usable
3. Leakage check	No source recording in more than one split	Passed; 1,000 unique recordings
4. Split construction	All three splits non-empty	9,750 / 650 / 2,600

Stage	Check	Observed result
5. Model and processor load	Weights, tokenizer, sampling rate	94,396,320 parameters; vocabulary 32; 16,000 Hz
6. Single sample	Dataset item shape	`input_values` [244560], `labels` [258]
7. Collation	Two samples batched	`input_values` [2, 244560] float32, `labels` [2, 258] int64
8. Training	Trainer runs to completion	2,000 steps, checkpoints and final model saved
9. Test prediction	Full test split decoded	2,600 predictions, WER 25.12%, CER 15.89%
10. Single-file inference	Transcribe one `.pt` file	Transcript returned (see below)

5.2 Optional subset smoke test

The notebooks ran on the full dataset. For a quick re-check (for example after changing the model or preprocessing), the same components can be run on 2 recordings (26 samples). The following snippet reuses the notebook's objects and was not executed for this report:

```
subset_ids = train_df["source_id"].drop_duplicates().iloc[:2]
subset = train_df[train_df["source_id"].isin(subset_ids)]        # 26 rows
ds = LibriSpeechDataset(subset, processor)
batch = data_collator([ds[i] for i in range(4)])
out = model(input_values=batch["input_values"].to(device), labels=batch["labels"].to(device))
print(out.loss, out.logits.shape)                                # finite loss, [4, T, 32]
print(processor.batch_decode(out.logits.argmax(-1).cpu()))
```

5.3 Model output examples

Single-file inference on the first test sample (clean):

- Reference: `AND AN EMBRYO COMPARED TO THE TWO COLOSSAL BARRICADES WHICH WE HAVE JUST SKETCHED BUT IT WAS FORMIDABLE FOR THAT EPOCH`
- Prediction: `AT AN EMBRYO COMPARED TO THE TWO COLOSSAL BARRICADES WHICH WE HAVE JUST SKETCHED BUT IT WAS FORMIDABLE FOR THAT EPOCH`

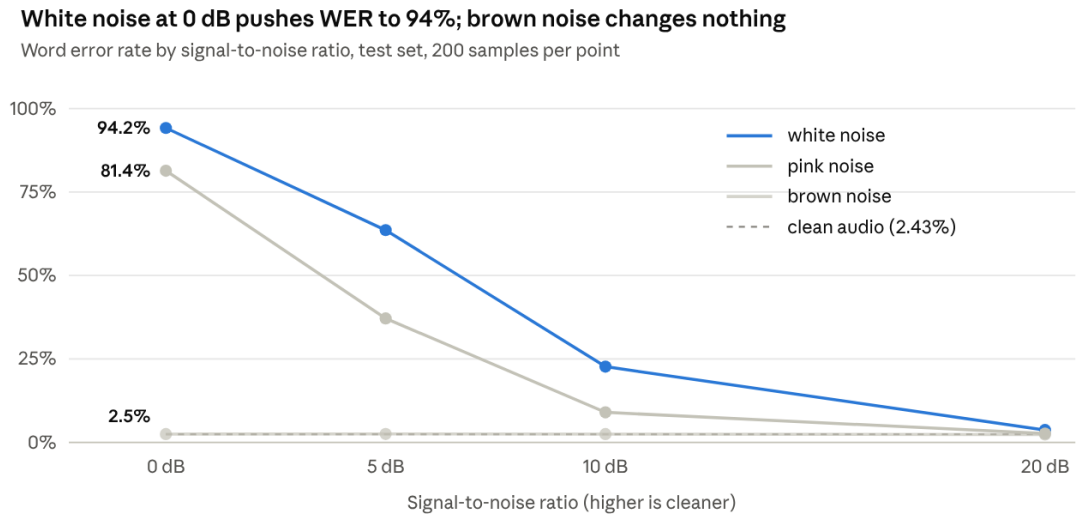
The only error is AND recognised as AT (one substitution).

Test predictions for one recording across noise conditions (first words only; the notebook table truncates long strings):

Condition	Prediction (start of utterance)
Reference	AND AN EMBRYO COMPARED TO THE TWO COLOSSAL BARRICADES …
Clean	AT AN EMBRYO COMPARED TO THE TWO COLOSSAL BARR…

Condition	Prediction (start of utterance)
White, 20 dB	AT AN EMBRYO COMPARED TO THE TWO COLOSSAL BARR…
White, 10 dB	AT AN IMBRYO COMPARITO THE TOO COLOSSAL VERITY…
White, 5 dB	AT AN EMBIO COMPERITY THE TO PFOF EVERITY I E…
White, 0 dB	AND IN ENVY O TE VERE TE IE TOO TEVERE TE TE E…
Pink, 0 dB	AND IN ENVY OFOPERIT TE FOFORVERITTO FO A YUS …
Brown, 0 dB	AT AN EMBRYO COMPARED TO THE TWO COLOSSAL BARR…

For a second recording, the reference begins I SUPPOSE A GOOD MANY OF US HAVE FELT JUST AS …; the 10 dB white-noise output is I SUPPOSE A GOOD MANY OF US TO FELT JUST TA JE…, and the 0 dB white-noise output collapses to BUT EO A ANIEEE EEO O O E E E E E O E I.



wav2vec2-training.ipynb, wer_by_noise_condition · 12 noise conditions plus clean, 200 test samples each

CER follows the same pattern, from 0.52% on clean audio to 73.09% for white noise at 0 dB. Hover a point for the WER and CER of each condition.

Three patterns stand out. Error grows steadily as SNR falls for white and pink noise, and white noise is harder than pink at every SNR. Brown noise has almost no effect, which is expected because its energy sits at very low frequencies, below most speech content, even at 0 dB. At 0 dB the white-noise output is no longer a transcript, which shows where the model needs the most improvement.

6. Loss and Evaluation

6.1 Loss function

Connectionist Temporal Classification (CTC) loss, computed inside `Wav2Vec2ForCTC` with `ctc_loss_reduction="mean"`. CTC sums the probability of every frame-level alignment that collapses to the target transcript (merge repeats, then remove blanks):

L = - log sum over alignments pi that map to y of the product over t of p(pi_t | x)

The `<pad>` token is the blank, label padding (`-100`) is masked out, and the per-sample loss is normalised by target length before averaging over the batch. CTC removes the need for frame-level labels, which we do not have. The same loss is reported as validation loss during evaluation.

6.2 Evaluation metrics

- **Word Error Rate (WER)** = (S + D + I) / N, where S, D and I are word substitutions, deletions and insertions and N is the number of reference words. It is the primary metric and the criterion for selecting the best checkpoint (lower is better).
- **Character Error Rate (CER)**, the same formula at the character level. It is more sensitive to partially correct words and so is informative at medium and low SNRs.
- **Per-condition WER and CER**, grouped by noise type and SNR, to measure robustness as noise increases.
- **Decoding:** greedy argmax decoding (no language model). Metrics are computed with the Hugging Face `evaluate` library (`wer`, `cer`, backed by `jiwer`).

Metrics planned for the word-level error detection component (not yet in the notebooks): precision, recall and F1 for flagging incorrectly recognised words from per-frame confidence.

7. Caveats and Next Steps

7.2 Other caveats

- **Pretraining overlap.** `wav2vec2-base-960h` was fine-tuned on all of LibriSpeech 960 h, which includes train-clean-100. Our clean WER (2.43%) is therefore optimistic, and gains should be judged mainly on noisy conditions.
- **Synthetic noise only.** All three noise types appear in train, validation and test, so the test set does not yet measure robustness to unseen noise. MUSAN and ESC-50 are the planned unseen and realistic noise sources. The EDA report text mentions on-the-fly augmentation and a brown-noise/ESC-50 test set, which does not match the implemented offline pipeline; the report text should be updated.
- **Correlated samples.** Each recording appears in 13 variants within its split, so the effective number of independent test utterances is 200, not 2,600.
- **EDA gap.** The LibriSpeech streaming EDA recorded duration statistics but its text statistics (character counts, speakers) failed, so they are not reported.

- **Length cap.** Audio is truncated to 30 s but the transcript is not. LibriSpeech utterances are shorter than this (maximum about 23 s in the EDA sample), so it has not triggered, but a guard would prevent misaligned labels.
- **Label normalisation.** Training labels are `text.upper()` from the dataset stage, while evaluation references use `normalized_text`; the two should be unified.

7.3 Next steps

1. Fix the training run (Section 7.1) and report fine-tuned against baseline WER per noise condition.
2. Add MUSAN and ESC-50 noise (the latter resampled from 44.1 kHz) with a noise-type-disjoint test split.
3. Benchmark WavLM and Whisper against the Wav2Vec 2.0 baseline.
4. Implement word-level error detection from CTC posteriors and evaluate it with precision, recall and F1.